

EXPERIMENT-7

Q1. Finding Angular Velocity when angular displacement and time are given.

Problem Statement:

Write a C program to find angular velocity when angular displacement and time are given.

(A)Understanding the Problem:

The program takes angular displacement and time as input and calculates angular velocity.

Input:

- Angular displacement (θ) in radians
- Time (t) in seconds

Output:

- Angular velocity (ω) in rad/s

Formula:

$$\omega = \theta / t$$

where:

- ω = Angular velocity
- θ = Angular displacement
- t = Time

Time must not be zero because division by zero is not possible.

(B)Test Case Table:

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	$\theta = 20$ rad, t = 4 s	5.00 rad/s	5.00 rad/s	Pass

2| $\theta = 45$ rad, $t = 5$ s| 9.00 rad/s| 9.00 rad/s| Pass
3| $\theta = 100$ rad, $t = 20$ s| 5.00 rad/s| 5.00 rad/s| Pass
4| $\theta = 12.5$ rad, $t = 2.5$ s| 5.00 rad/s| 5.00 rad/s| Pass
5| $\theta = 60$ rad, $t = 0$ s| Invalid input| Invalid input| Pass

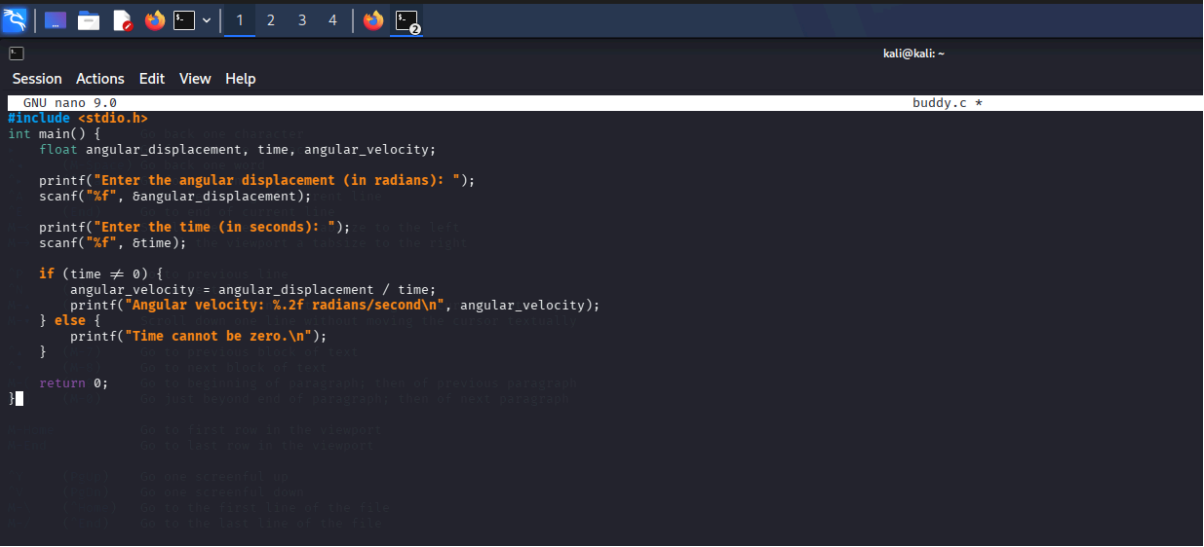
(C)Algorithm:

1. Start.
2. Declare three variables: angular displacement, time, and angular velocity.
3. Ask the user to enter angular displacement in radians.
4. Read the angular displacement.
5. Ask the user to enter time in seconds.
6. Read the time.
7. Check whether time is zero.
8. If time is zero, display an invalid input message.
9. Otherwise, calculate angular velocity by dividing angular displacement by time.
10. Display the angular velocity.
11. Stop.

Algorithm Evaluation:

The algorithm was checked against all the test cases in the table. For valid values of time, it correctly calculates angular velocity. It also handles the case where time is zero and prevents division by zero. Therefore, the algorithm works correctly for the prepared test cases.

(E) Program, Execution and Evaluation:



The screenshot shows a terminal window with the nano text editor open. The editor displays a C program named 'buddy.c'. The program prompts the user to enter angular displacement and time, then calculates and prints the angular velocity. The terminal output shows the program being executed successfully with the input values 20 and 4, resulting in an output of 5.00 rad/s.

```
GNU nano 9.0 buddy.c *
#include <stdio.h>
int main() {
    float angular_displacement, time, angular_velocity;

    printf("Enter the angular displacement (in radians): ");
    scanf("%f", &angular_displacement);

    printf("Enter the time (in seconds): ");
    scanf("%f", &time);

    if (time != 0) {
        angular_velocity = angular_displacement / time;
        printf("Angular velocity: %.2f radians/second\n", angular_velocity);
    } else {
        printf("Time cannot be zero.\n");
    }

    return 0;
}
```

Sample Execution:

Enter angular displacement in radians: 20

Enter time in seconds: 4

Angular velocity = 5.00 rad/s

The program was evaluated using the test cases prepared in Step 2.

The actual outputs matched the expected outputs, so all the test cases passed.

Q2. Finding shear modulus when shear stress and shear strain are given.

Problem Statement:

Write a C program to find shear modulus when shear stress and shear strain are given.

(A) Understanding the Problem:

The program takes shear stress and shear strain as input and calculates the shear modulus of the material.

Input:

- Shear stress (τ) in Pa
- Shear strain (γ)

Output:

- Shear modulus (G) in Pa

Formula:

$$G = \tau / \gamma$$

where:

- G = Shear modulus
- τ = Shear stress
- γ = Shear strain

Shear strain must not be zero because division by zero is not possible.

(B) Test Case Table:

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	Stress = 100 Pa, Strain = 0.5	200.00 Pa	200.00 Pa	Pass
2	Stress = 500 Pa, Strain = 0.25	2000.00 Pa	2000.00 Pa	Pass
3	Stress = 1200 Pa, Strain = 0.4	3000.00 Pa	3000.00 Pa	Pass
4	Stress = 750 Pa, Strain = 0.5	1500.00 Pa	1500.00 Pa	Pass
5	Stress = 500 Pa, Strain = 0	Invalid input	Invalid input	Pass

(C) Algorithm:

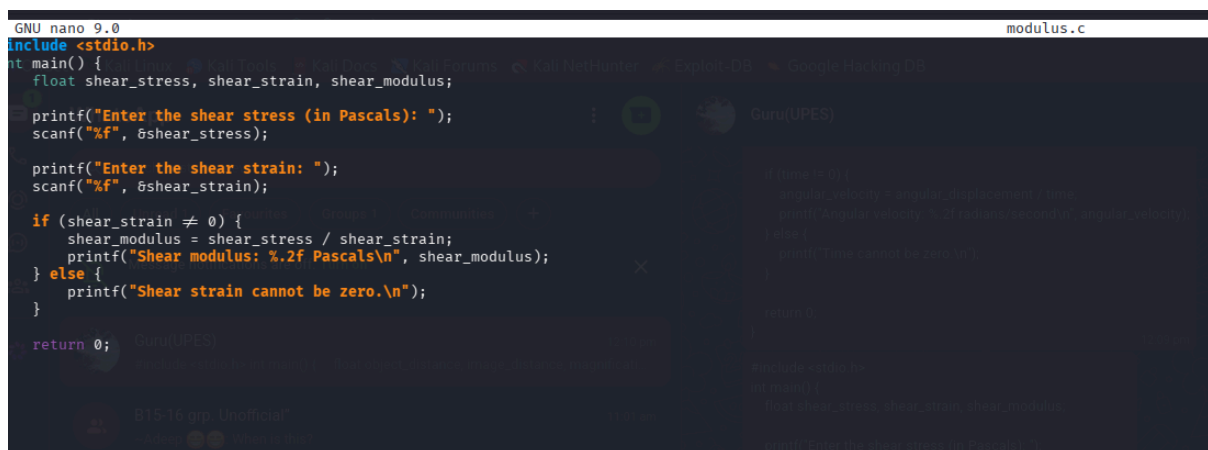
1. Start.
2. Declare three variables: shear stress, shear strain, and shear modulus.
3. Ask the user to enter shear stress.
4. Read the shear stress.
5. Ask the user to enter shear strain.
6. Read the shear strain.
7. Check whether shear strain is zero.
8. If shear strain is zero, display an invalid input message.

9. Otherwise, calculate shear modulus by dividing shear stress by shear strain.
10. Display the shear modulus.
11. Stop.

Algorithm Evaluation:

The algorithm was checked using all the prepared test cases. It correctly calculates shear modulus when the shear strain is non-zero. It also handles zero shear strain and prevents division by zero. Therefore, the algorithm successfully passes the prepared test cases.

(D) Program, Execution and Evaluation:



```
GNU nano 9.0 modulus.c
#include <stdio.h>
int main() {
    float shear_stress, shear_strain, shear_modulus;

    printf("Enter the shear stress (in Pascals): ");
    scanf("%f", &shear_stress);

    printf("Enter the shear strain: ");
    scanf("%f", &shear_strain);

    if (shear_strain != 0) {
        shear_modulus = shear_stress / shear_strain;
        printf("Shear modulus: %.2f Pascals\n", shear_modulus);
    } else {
        printf("Shear strain cannot be zero.\n");
    }

    return 0;
}
```

Sample Execution:

Enter shear stress in Pa: 100
Enter shear strain: 0.5
Shear modulus = 200.00 Pa

The program was evaluated using the test cases from Step 2. The calculated outputs matched the expected outputs for all the prepared cases.

Q3. finding mirror magnification when image and object distance are given.

Problem Statement:

Write a C program to find mirror magnification when image distance and object distance are given.

(A)Understanding the Problem:

The program takes image distance and object distance as input and calculates the magnification produced by a mirror.

Input:

- Image distance (v)
- Object distance (u)

Output:

- Mirror magnification (m)

Formula:

$$m = -v / u$$

where:

- m = Magnification
- v = Image distance
- u = Object distance

The negative sign is included according to the standard mirror magnification formula. Object distance must not be zero.

(B)Test Case Table:

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	v = 20 cm, u = 10 cm	-2.00	-2.00	Pass
2	v = 30 cm, u = 15 cm	-2.00	-2.00	Pass
3	v = 15 cm, u = 30 cm	-0.50	-0.50	Pass
4	v = -20 cm, u = 40 cm	0.50	0.50	Pass
5	v = 20 cm, u = 0 cm	Invalid input	Invalid input	Pass

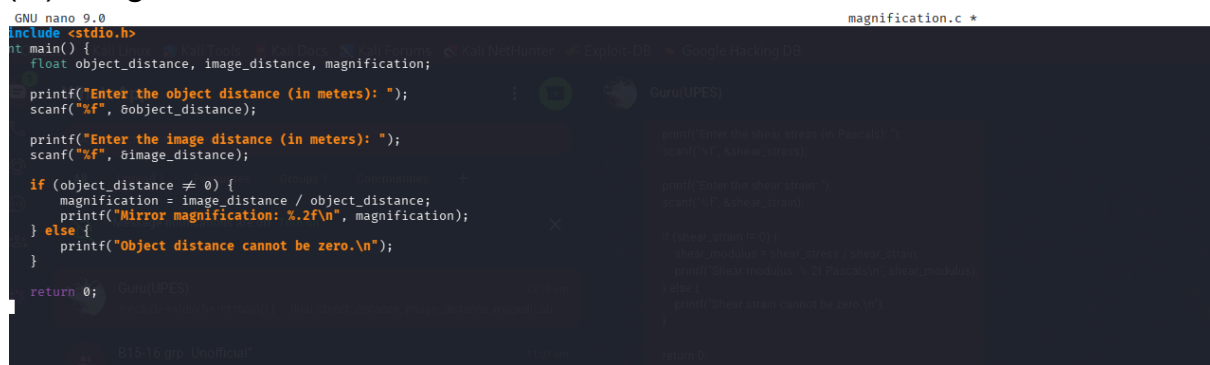
(C)Algorithm:

1. Start.
2. Declare three variables: image distance, object distance, and magnification.
3. Ask the user to enter the image distance.
4. Read the image distance.
5. Ask the user to enter the object distance.
6. Read the object distance.
7. Check whether object distance is zero.
8. If object distance is zero, display an invalid input message.
9. Otherwise, divide the negative of image distance by object distance to calculate magnification.
10. Display the magnification.
11. Stop.

Algorithm Evaluation:

The algorithm was evaluated using all the prepared test cases. It correctly calculates mirror magnification for valid image and object distances. It also handles the zero object-distance case to prevent division by zero. Hence, the algorithm passes the prepared test cases.

(D) Program, Execution and Evaluation:



```
GNU nano 9.0 magnification.c *
#include <stdio.h>
int main() {
    float object_distance, image_distance, magnification;

    printf("Enter the object distance (in meters): ");
    scanf("%f", &object_distance);

    printf("Enter the image distance (in meters): ");
    scanf("%f", &image_distance);

    if (object_distance != 0) {
        magnification = image_distance / object_distance;
        printf("Mirror magnification: %.2f\n", magnification);
    } else {
        printf("Object distance cannot be zero.\n");
    }

    return 0;
}
```

Sample Execution:

Enter image distance in cm: 20
Enter object distance in cm: 10
Mirror magnification = -2.00

The program was evaluated against the test cases prepared in Step 2.
The actual results matched the expected results, so the program passed all the prepared test cases.

NAME: TAANISH AGARWAL

BATCH: B-15

SAP ID: 590034662